Supplementary Document

Use LLMs to Build Repair Tools, Not Patch Generation: Cost-Effective Automated Program Repair

Anonymous submission

This document carries what the main paper's page budget could not. Nothing here introduces a claim the paper does not already make: each item is the tabular or graphical form of a number the Results section gives, or a measurement the paper cites without room to print. Assignment ids (R1–R5, C1–C6), the arm names, and every metric definition are the main paper's. Values are generated from `results/` by the released scripts (`run.py tables`, `run.py figures`); none is transcribed.

Where the Repairs Come From

Table S1 decomposes every repair by the strategy that produced it, with the mean patch each leaves and whether it carries a misconception label, pooled over both builders. Shares are over repaired submissions; the LLM row belongs to CoEP_{LF} and sits outside that denominator. The last rows are the framework-only ablation, the identical tool with the builder's rules removed, which is the same artifact under either builder because nothing the model wrote survives in it.

Repair source	share (%)	mean RPS	labelled
1. rules	24.6	0.100	•
2. reference transplant	75.4	0.581	–
3. LLM (CoEP_{LF})	8.2	0.724	–
Framework-only ablation (builder rules removed), first column is RR (%) rather than share			
ablated	98.69	0.492	–
full tool	99.06	0.463	–

Table S1: Repair source, patch size, and labelling, with the framework-only ablation.

What Escalation Costs, Assignment by Assignment

The main paper reports the escalation rate e as a corpus figure. It is not uniform: Table S2 gives it per assignment alongside the calls it implies, the local CPU each submission spends, and the one-off build wall clock. Two readings matter for a department sizing this. The assignments that escalate most are the ones whose local patches are largest rather than the hardest ones, and the compute is not free: 5,940 submissions cost 4.8 core hours of local search, on top of roughly 22 minutes of build per assignment per builder. Neither figure appears in the dollar totals, which price model tokens alone.

	e (%)	calls	ATT_{cpu} (s)	build (min)
R1	0.7	8	2.28	32
R2	22.5	196	3.18	27
R3	0.0	0	1.83	18
R4	9.2	66	3.43	28
R5	0.9	2	4.79	18
C1	13.2	116	3.17	20
C2	23.3	261	2.22	38
C3	13.0	18	6.94	3
C4	5.3	4	1.34	12
C5	15.6	24	4.86	4
C6	66.7	4	6.58	6
All	**11.8**	**699**	**2.88**	**206**

Table S2: Escalation rate, calls, local CPU per submission, and build wall clock per assignment, over both builders.

Robustness

Table S3 puts three axes side by side: assignment difficulty, the choice of builder, and the escalation threshold. The difficulty bands group the eleven assignments as CS1, Codeforces 800–1000, and Codeforces 1200–2000, the last folding C6's three submissions in rather than reporting them alone.

Figure S1 plots every measured threshold setting against the baselines it is compared to. The point is the emptiness between the cluster and the baselines: the whole family spans \$12.47–\$15.61 of total spend, so which setting ships cannot change a conclusion. The shipped rule, escalate a passing local patch whose RPS exceeds 1, is marked with a star and is the canonical run rather than the sweep's reconstruction of it.

Rule Provenance

The builders wrote 127 rules that fired, 1,216 times between them, against the five always-on rules that fired 631 times. The distinction that matters is reuse. Exactly one builder-written rule fired on more than

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Stratum	Refactory	PyDex RR (%) / RPS	CoEP_{LF}
Difficulty (`claude-sonnet-5`)			
CS1	90.8 / 0.397	98.6 / 0.382	99.4 / 0.279
CF 800–1000	83.1 / 0.663	99.4 / 0.942	100.0 / 0.572
CF 1200–2000	54.5 / 0.815	100.0 / 0.800	100.0 / 0.693
Builder			
`claude-sonnet-5`	–	99.0 / 0.598	99.7 / 0.404
`gpt-5.4-mini`	–	98.9 / 0.669	99.7 / 0.417
Threshold, 29 measured settings			
range	–	–	99.7 / 0.378–0.451
spend		$12.47–$15.61 total	

Table S3: Difficulty, builder, and escalation threshold.

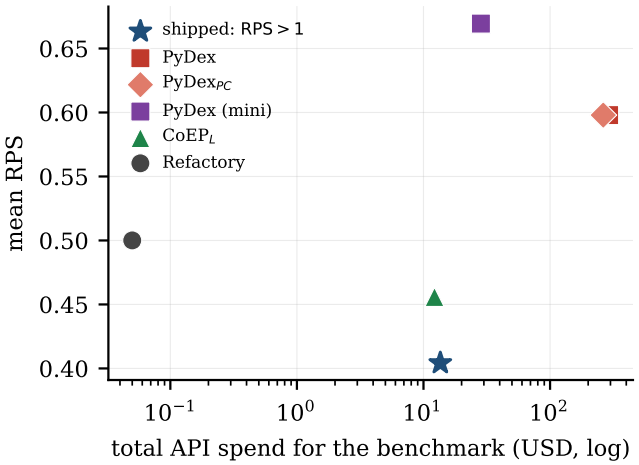


Figure S1: Mean RPS against total spend. Each small marker is one threshold setting; the star is the shipped rule. Repair rate is identical across all of them.

one assignment, `add_missing_call_parens`, while `generic_flip_compare` alone fired 455 times across 10 of the 11 assignments. The authored layer is assignment-specific by construction, which is why it is priced per assignment and the always-on layer is not.

The rules that carry the most submissions read as misconceptions rather than as patches: `slice_case_check_to_equality` (115 repairs on C2), `append_missing_return_len_after_loop` (114 on R1), `count_compare_relax` (112 on R2), and `strict_one_should_be_exactly_one` (96 on R2). On C4 a single rule, `carry_fix_two_value_print`, repairs 27 of the assignment's 38 submissions under `claude-sonnet-5`.

The eight rules self-learning absorbed read the same way: `counter_var_confusion`, `uniqueness_count_gt1_to_neq1`, `index_vs_count_and_uncalled_bool` and `missing_else_false` on R2 with `claude-sonnet-5`; `count_equality_for_uniqueness`, `count_uniqueness_instead_of_direct_membership` and `missing_call_and_predicate_fix` on R2 with `gpt-5.4-mini`; and `binarysearch_zero_exit` on C3.

Reproducibility

The repository contains the harness, every prompt, the 22 synthesized repairers with their build costs, and the per-submission event log each arm wrote. Four commands regenerate every number in both documents from those logs without touching a model: `run.py tables`, `run.py figures`, `run.py numbers`, which recomputes each quantity and reports whether the string still occurs in the paper, and `run.py stats`. The measured threshold settings are archived as `results/threshold_sweep.json`.

Two dependencies are outside our control and worth stating. Both builders are closed commodity models reached through their providers' APIs, so a rebuild cannot be guaranteed to reproduce the same rules; what is released is the tools those builds produced, which are ordinary Python and run offline. Refactory's replication package is its authors' and is cloned rather than vendored.